



Module 15-1

Designing Finite State Machines (FSMs)

cādence®

This module more specifically presents the various ways to code a state machine for synthesis.

Module Objective

In this module, you:

- Code state machines for synthesis

Topics

- FSM introduction
- Defining the FSM states
- Example read-write synchronizer FSM
- Coding FSMs in various styles
- Various ways to optimize FSMs



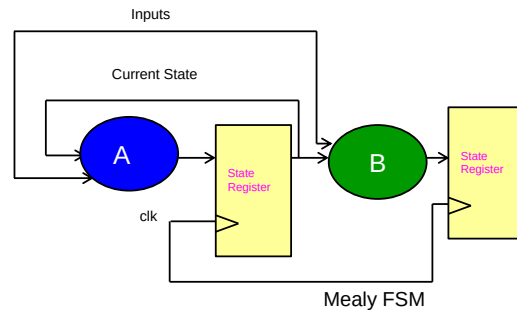
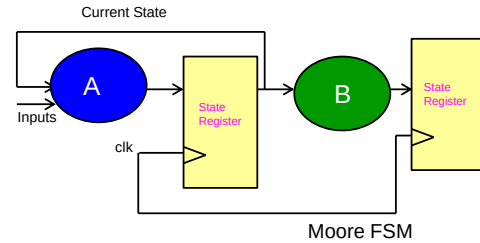
Your objective is to code state machines for synthesis.

To help you do that, this module discusses what an FSM is, how to define FSM states, various ways to code FSMs for synthesis, and various ways to optimize FSMs.

Finite State Machine (FSM) Review

FSM Structure

- **State Register**
 - Stores current state
- **Next State decode logic**
 - Decides next state based on current state and inputs
- **Output Logic**
 - Decodes state (or states and inputs) to produce outputs
- Outputs from the FSM can be a function of:
 - Current state only – **Moore**
 - Current state and the current inputs – **Mealy**



An FSM consists of state-encoding combinational logic, state-storing sequential elements, and output-decoding combinational logic.

- A Moore machine has no combinational path from inputs to outputs. The outputs are a function solely of the current state vector.
- A Mealy machine has at least one combinational from an input to an output. The outputs are a function of the current state and at least one current input. Mealy outputs can be available up to one clock earlier path than Moore outputs, but may complicate synthesis timing constraint definition.

If the current state of your design depends at least partially upon the previous state, then your design is an FSM.

Perhaps one way to remember which is the Mealy machine and which is the Moore machine is the phrase “Mealy is more and Moore is less”. This refers to the outputs, which in the Mealy machine can include unregistered inputs.

Defining the FSM States

You will likely see FSM descriptions that define state vector values with text replacement macros. This training does not do that and you should neither.

Macros

```
`define IDLE 2'd0
`define READ 2'd1
`define WRITE 2'd2
`define DONE 2'd3
reg [1:0] state, nstate;
```

```
always @*
case ( state )
  `IDLE: nstate = do_write ? `WRITE
                        : `READ;
...

```

- Scope is global – across files and modules from `define to `undef
- Accepted by synthesis tools but not for FSM optimization

Parameters

```
localparam IDLE = 2'd0,
            READ = 2'd1,
            WRITE = 2'd2,
            DONE = 2'd3;
reg [1:0] state, nstate;
```

```
always @*
case ( state )
  IDLE: nstate = do_write ? WRITE
                        : READ;
...

```

- Scope is local to declaring block
- Required by synthesis tools that perform FSM optimizations



The scope of any compiler directive is from the point of the direction to the point of its redirection or removal, potentially across multiple files and multiple modules. To inadvertently use a macro defined elsewhere or to define a macro inadvertently used elsewhere is very easy. To help prevent such erroneous use, designers typically place text macro definitions in a separate file that any compilation unit that uses the definitions includes during compilation. This encapsulates those definitions in a single easily-modified place.

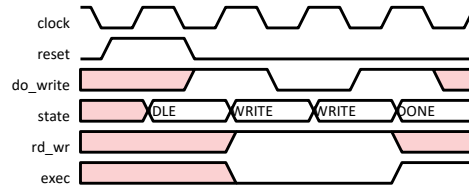
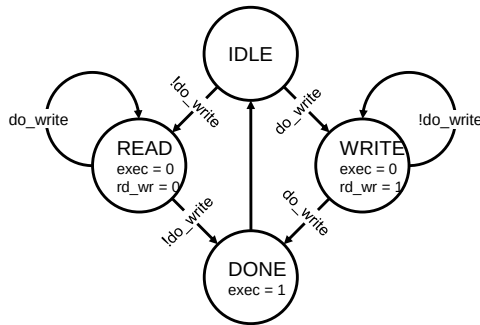
The scope of a local parameter is the declaring block. No code can change its initialized value.

Synthesis tools that perform state machine optimizations require you to define the states with parameters if you want the tool to perform the optimizations.

Example Read-Write Synchronizer FSM

The following examples refer to this FSM specification:

- If `do_write` is true, transition to `WRITE` and set `exec` = 0 and `rd_wr` = 1.
 - When `do_write` is again true, transition to `DONE` and set `exec` = 1.
- If `do_write` is false, transition to `READ` and set `exec` = 0 and `rd_wr` = 0.
 - When `do_write` is again false, transition to `DONE` and set `exec` = 1.

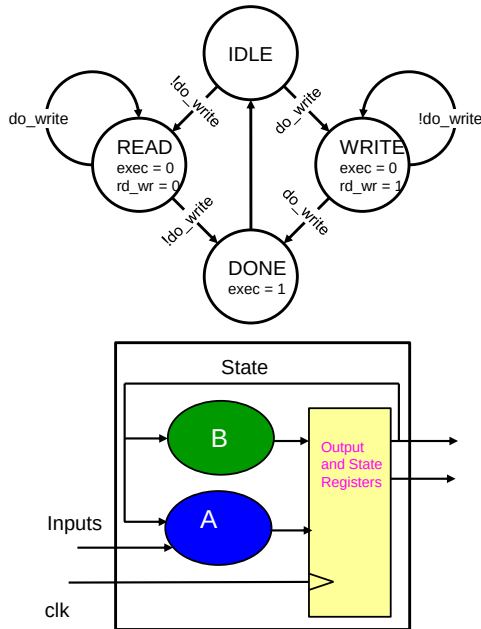


Outputs(`exec` and `rd_wr`) are considered unknown at reset for simplicity, in reality it may be either 1 or 0.

The following examples refer to this state machine specification:

- If `do_write` is true, it transitions to the `WRITE` state and sets `exec` to 0 and `rd_wr` to 1. When `do_write` is again true, it transitions to the `DONE` state and sets `exec` to 1.
- If `do_write` is false, it transitions to the `READ` state and sets `exec` to 0 and `rd_wr` to 0. When `do_write` is again false, it transitions to the `DONE` state and sets `exec` to 1.

Coding the FSM in One Block: Sequential Outputs



```

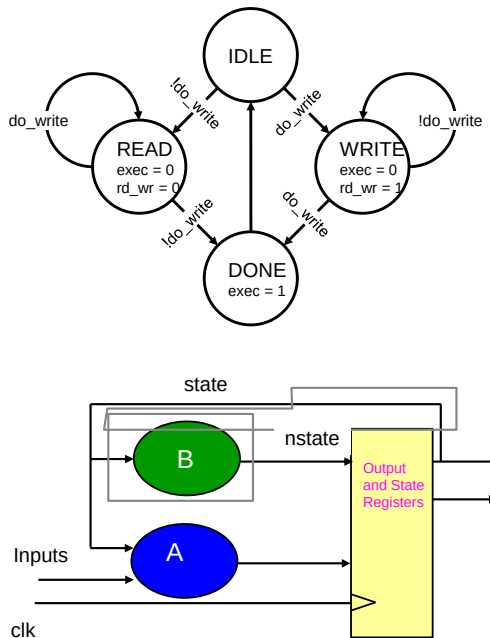
localparam IDLE = 2'd0, READ = 2'd1,
            WRITE = 2'd2, DONE = 2'd3;
reg [1:0] state;
reg exec, rd_wr;

always @(posedge clock)
if ( reset )
    state <= IDLE;
else
    case ( state )
        IDLE: begin
            exec <= 0;
            rd_wr <= do_write;
            state <= do_write ? WRITE
                           : READ;
        end
        READ: if ( !do_write )
            {state,exec}<={DONE,1'b1};
        WRITE: if ( do_write )
            {state,exec}<={DONE,1'b1};
        DONE: state <= IDLE;
    endcase

```

The one-block coding style generates the next-state, state, and outputs all in one sequential block.

Coding the FSM in Two Blocks: Sequential Outputs



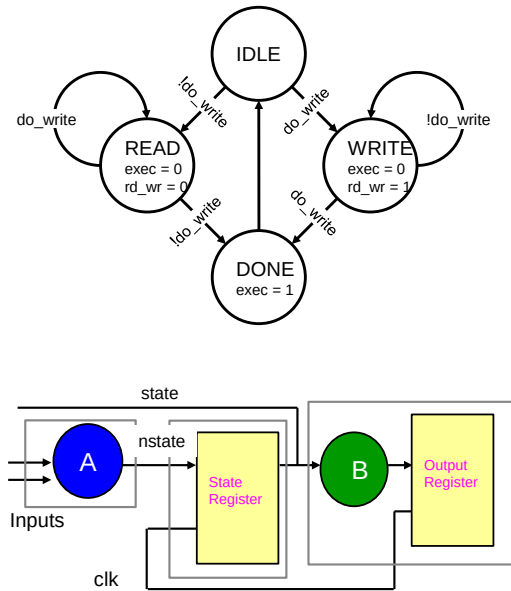
```
localparam IDLE = 2'd0, READ = 2'd1,
           WRITE = 2'd2, DONE = 2'd3;
reg [1:0] state, nstate;
reg exec, rd_wr;

always @*
case ( state )
  IDLE: nstate= do_write? WRITE: READ;
  READ: nstate= !do_write? DONE : READ;
  WRITE: nstate= do_write? DONE : WRITE;
  DONE: nstate= IDLE;
endcase

always @(posedge clock)
if ( reset )
  state <= IDLE;
else
begin
  state <= nstate;
  case ( nstate )
    READ: {rd_wr,exec} <= 2'b00;
    WRITE: {rd_wr,exec} <= 2'b10;
    DONE: exec <= 1;
  endcase
end
```

The two-block coding style encodes the next state in a separate combinational block. You must place asynchronous reset in the sequential block. You can place synchronous reset in either block, but synthesis tools might more accurately identify the component's reset pin if you place the reset in the sequential block.

Coding the FSM in Three Blocks: Sequential Outputs



```

localparam IDLE = 2'd0, READ = 2'd1,
            WRITE = 2'd2, DONE = 2'd3;
reg [1:0] state, nstate;
reg exec, rd_wr;

always @*
case ( state )
  IDLE: nstate= do_write? WRITE: READ;
  READ: nstate= !do_write? DONE : READ;
  WRITE: nstate= do_write? DONE : WRITE;
  DONE: nstate= IDLE;
endcase

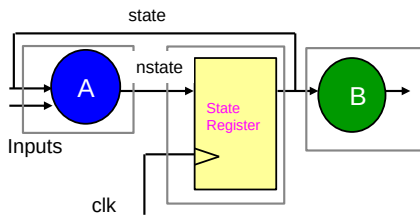
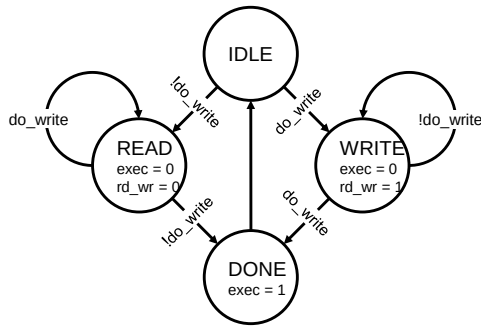
always @(posedge clock)
if ( reset )
  state <= IDLE;
else
  state <= nstate;

always @(posedge clock)
if ( !reset )
  case ( nstate )
    READ: {rd_wr,exec} <= 2'b00;
    WRITE: {rd_wr,exec} <= 2'b10;
    DONE: exec <= 1;
  endcase

```

The three-block coding style decodes the outputs in a third block. Here, that third block is sequential. Note that the outputs do not transition while the reset is active.

Coding in Three Blocks: Combinational Outputs



```

localparam IDLE = 2'd0, READ = 2'd1,
            WRITE = 2'd2, DONE = 2'd3;
reg [1:0] state, nstate;
reg exec, rd_wr;

always @*
case ( state )
  IDLE: nstate= do_write? WRITE: READ;
  READ: nstate= !do_write? DONE : READ;
  WRITE: nstate= do_write? DONE : WRITE;
  DONE: nstate= IDLE;
endcase

always @(posedge clock)
if ( reset )
  state <= IDLE;
else
  state <= nstate;

always @*
case ( state )
  IDLE: {rd_wr,exec} = 2'bxx;
  READ: {rd_wr,exec} = 2'b00;
  WRITE: {rd_wr,exec} = 2'b10;
  DONE: {rd_wr,exec} = 2'b11;
endcase
    
```

The three-block coding style decodes the outputs in a third block. Here, that third block is combinational. The combinational block takes advantage of the specification that for some states we do not care what the output value is.

Optimizing Register Count

- Registering outputs is preferable for synthesis as it simplifies timing constraints.
- Examine the mapping between state and output.
- Can output registers replace state bits? If so then you can reduce your overall register count.

State	Encoding	rd_wr	exec
IDLE	01	?	?
READ	00	0	0
WRITE	10	1	0
DONE	11	?	1
match:		state[1]	state[0]

```
localparam READ = 2'd0, IDLE = 2'd1,
            WRITE = 2'd2, DONE = 2'd3;
reg [1:0] state, nstate;
reg exec, rd_wr;

always @*
case ( state )
  IDLE: nstate= do_write? WRITE: READ;
  READ: nstate= !do_write? DONE : READ;
  WRITE: nstate= do_write? DONE : WRITE;
  DONE: nstate= IDLE;
endcase

always @(posedge clock)
if ( reset )
  state <= IDLE;
else
  state <= nstate;

always @*
begin
  rd_wr = state[1];
  exec = state[0];
end
```

This page does not contain notes.

Optimizing Power and Noise: Gray Encoding

Where vectors regularly traverse a range of values, opportunities exist to encode values in consideration of power efficiency and noise reduction. Perhaps the most classic of examples is Gray-encoded counting.

Binary	Transitions
00	2
01	1
10	2
11	1

mean: 1.5

Gray	Transitions
00	1
01	1
11	1
10	1

mean: 1

Gray code effective for predictable state patterns with a specific number of state transitions.

State	Transitions
(00)IDLE	2
(01)READ	1
(11)DONE	1

mean: 1.3

State	Transitions
(00)IDLE	2
(10)WRITE	1
(11)DONE	1

mean: 1.3

11 © Cadence Design Systems, Inc. All rights reserved.



Bell Labs researcher Frank Gray applied for patent 2,632,058 “Pulse Code Communication” in 1947, defining a “reflected binary code” for lack of a better term. Coincidentally with the patent award, the codes became popularly known as Gray codes. A Gray code is “a binary numeral system where two successive values differ in only one digit” – http://en.wikipedia.org/wiki/Gray_code

The tabulated sequence is the original “binary reflected Gray code” (that you can generate from binary using $g=b^{(b \gg 1)}$) but other Gray codes also exist.

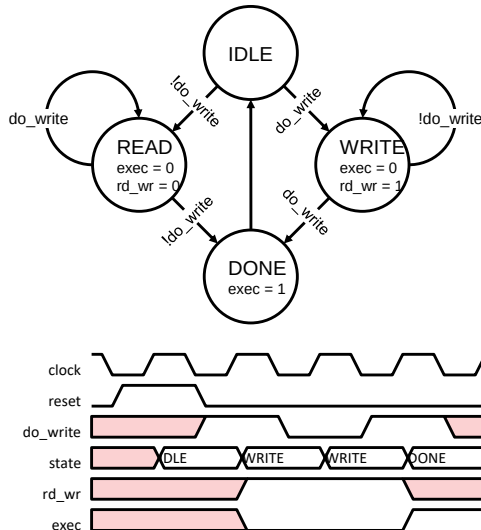
While initially used to work around switch-bounce, as the new vector value can be used immediately without further waiting for the value to “settle”, gray-encoding is indispensable for address counters that increment in one clock domain and are read in another, for example in asynchronous FIFOs.

Gray-encoding has more recently become valuable in the quest for reduced switching activity. The simple counting sequence displayed here averages 1.5 transitions per clock for binary encoding and 1 transition per clock for Gray encoding.

Paths through the example machine have a length of only three states, so present little opportunity to reduce switching activity.

Optimizing Performance: One-Hot Encoding

Reducing state encoding/decoding shortens timing paths to increase performance.



```
localparam IDLE = 4'd1, READ = 4'd2,
            WRITE = 4'd4, DONE = 4'd8;
reg [3:0] state, nstate;
reg exec, rd_wr;

always @*
case ( state )
  IDLE: nstate= do_write? WRITE: READ;
  READ: nstate= !do_write? DONE : READ;
  WRITE: nstate= do_write? DONE : WRITE;
  DONE: nstate= IDLE;
endcase

always @(posedge clock)
if ( reset )
  state <= IDLE;
else
  state <= nstate;

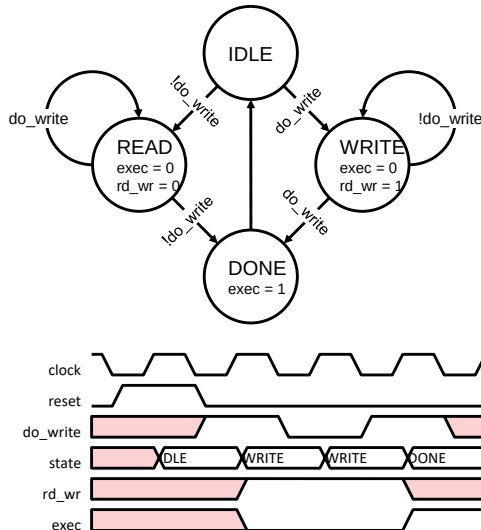
always @*
begin
  rd_wr = state == WRITE;
  exec = state == DONE;
end
```

One-Hot is effective for any number of arbitrary transition patterns.

One-hot state encoding provides a unique state bit for each state. This reduces the combinational logic required to encode and decode the state, thus shortens the timing path to permit increased clock speed.

One-Hot Encoding by Indexing State Bits

Indexing state bits more clearly reveals the reduced encoding.



```
localparam IDLE = 0, READ = 1,
           WRITE = 2, DONE = 3;
reg [3:0] state, nstate;
reg exec, rd_wr;

always @*
begin
    nstate[READ] = state[IDLE] && !do_write
                || state[READ] && do_write;
    nstate[WRITE] = state[IDLE] && do_write
                  || state[WRITE] && !do_write;
    nstate[DONE] = state[READ] && !do_write
                  || state[WRITE] && do_write;
    nstate[IDLE] = state[DONE];
end

always @(posedge clock)
begin
    state[IDLE] <= reset || nstate[IDLE];
    state[READ] <= !reset && nstate[READ];
    state[WRITE] <= !reset && nstate[WRITE];
    state[DONE] <= !reset && nstate[DONE];
end

always @*
begin
    rd_wr = state[WRITE];
    exec = state[DONE];
end
```

You can code the one-hot machine in a style that individually references each state bit. This style may more clearly reveal the reduced state encoding. Here, the longest path to any state bit is a sum of two products of three terms each.

Module Summary

You should now be able to code state machines for synthesis.

This module discussed:

- What is a FSM
 - State encoding, storage, and decoding
- How to define FSM states
 - By use of local constants
- Various ways to code FSMs for synthesis
 - 1-block, 2-block, 3-block
- Various ways to optimize FSMs
 - For area, power, noise, performance



You should now be able to code state machines for synthesis.

This module discussed what an FSM is, how to define FSM states, various ways to code FSMs for synthesis, and various ways to optimize FSMs.

Module Review

1. What is the difference between a Moore machine and a Mealy machine?
2. Why should you define state values using parameters instead of macros?
3. In how many blocks should you define the FSM to generate the best quality netlist?



This page does not contain notes.

Module Review Solutions

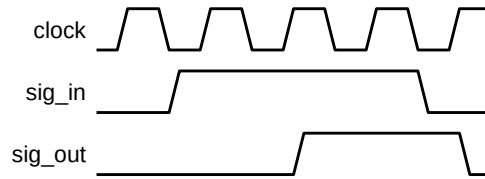
1. What is the difference between a Moore machine and a Mealy machine?
 - The outputs of a Moore machine are a function purely of the machine state. The outputs of a Mealy machine are a function partially of at least one machine input.
2. Why should you define state values using parameters instead of macros?
 - A synthesis tool is more likely to recognize a state machine if you define state values using parameters instead of macros. Recognizing the state machine permits optimizations such as Gray or one-hot encoding.
3. In how many blocks should you define the FSM to generate the best quality netlist?
 - The netlist quality does not generally depend upon the number of blocks. Choose a number that clearly communicates your design intention.



This page does not contain notes.

Module Exercise

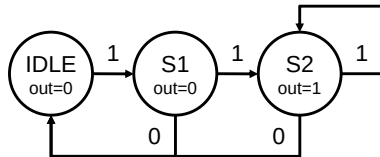
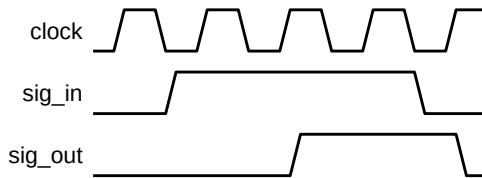
Architect and code a FSM that outputs a “one” for one clock whenever it recognizes two adjacent ones in the input stream.



This page does not contain notes.

Module Exercise Solutions

Architect and code a FSM that outputs a “one” for one clock whenever it recognizes two adjacent ones in the input stream.



```
module fsm
(
  input wire clock, reset, sig_in,
  output wire sig_out
);
  reg [1:0] state;
  localparam IDLE=0, S1=1, S2=2;
  always @(posedge clock)
  if ( reset )
    state <= IDLE;
  else
    case ( state )
      IDLE: state <= sig_in ? S1 : IDLE;
      S1,S2: state <= sig_in ? S2 : IDLE;
    endcase
  assign sig_out = (state==S2);
endmodule
```

This page does not contain notes.



Module 15-2

Avoiding Simulation Mismatches

cā dence®

This module addresses the common and perplexing problem of the post-synthesis simulation results that do not match the pre-synthesis simulation results.

Module Objective

In this module, you:

- Avoid the common causes of mismatches between RTL simulation and post-synthesis netlist simulation

Topics

- Non-deterministic behavior
- Conditional compilation
- Incomplete sensitivity list
- Temporary variables
- Asynchronous set and reset
- RTL vs. Synthesis Models
- Incomplete Assignments
- Comparing unknown values
- Case statements: **casex** and **casez**
- Variable declaration assignment
- Delay controls



This page does not contain notes.

Avoiding Indeterminate Behavior

You have already seen that the Verilog standard permits the simulator to execute triggered procedures in any order.

The Verilog standard also permits the simulator to execute contiguous statements as multiple events, thus potentially interleaving statement execution from multiple blocks!

```
always @(posedge clk)
begin
    b = a;
    c = b;
end

always @(posedge clk)
begin
    d = b ^ c;
end
```

Potential Sequences

```
b = a;
c = b;
d = b ^ c;
```

```
d = b ^ c;
b = a;
c = b;
```

```
b = a;
d = b ^ c;
c = b;
```



Make only nonblocking assignments to inter-procedure signals.

You have already seen that the Verilog standard permits the simulator to execute triggered procedures in any order.

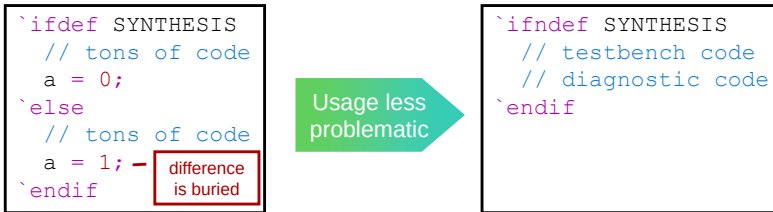
The Verilog standard also permits the simulator to execute contiguous statements as multiple events, thus potentially interleaving statement execution from multiple blocks!

You can greatly reduce indeterminacy by making only nonblocking assignments to storage variables that communicate between procedures.

“At any time while evaluating a behavioral statement, the simulator may suspend execution and place the partially completed event as a pending active event on the event queue. The effect of this is to allow the interleaving of process execution. Note that the order of interleaved execution is nondeterministic and not under control of the user.” – IEEE Std. 1364-2001 Section 5.4.2 Nondeterminism

Applying Conditional Compilation Carefully

Carefully consider what code you “hide” from simulation or synthesis!

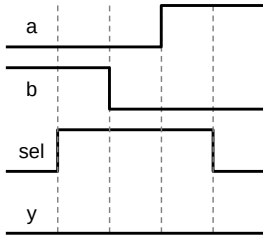


Carefully consider what code you hide from simulation or synthesis. You can reasonably safely hide from synthesis testbench-related code that you embed in an RTL module. Much less valid reason exists to hide design code from simulation. Complex, frequent and large conditionally compiled regions especially promote inadvertent coding errors that RTL simulation does not encounter.

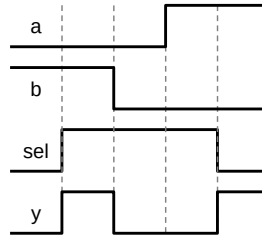
Completing the Combinational Logic Sensitivity List

For simulation, include in the event list all signals that are input to the logic. For your convenience, you can use the “*” wildcard.

```
// Incomplete list
always @(a or b)
begin
    y = a;
    if (sel)
        y = b;
end
```



```
// Complete list
always @*
begin
    y = a;
    if (sel)
        y = b;
end
```



23 © Cadence Design Systems, Inc. All rights reserved.



The event list does not affect the synthesis result, but to avoid incorrect RTL simulation results, you should include in the event list all inputs to the procedure. The easiest way to ensure this is to use the Verilog 2001 wildcard event control (@*).

This example illustrates the effect of an incomplete sensitivity list. If the event list omits the sel signal, the procedure executes upon transitions of only the a and b inputs – transitions of the sel signal have no affect.

Debugging problems caused by an incomplete sensitivity list is difficult, so you might want to develop the habit of simply always using the wildcard event control for all combinational procedures.

“The event list does not affect the synthesized netlist.” – IEEE Std. 1364.1-2002 5.1 Modeling combinational logic

Ensuring Temporary Variables Are Truly Temporary

- Temporary variables are those written and then read in the same procedure and nowhere else.
- Here, “temp” is not a temporary variable so must appear in the event list.
- The event list for a combinational procedure should contain all inputs to the logic.
- Do not include temporary variables – those written and then read in the same procedure and nowhere else.

```
// Combinational logic

always @(a or b or c)
begin : comb_blk
    reg temp;
    temp = a + b;
    q = temp + c;
end
```

```
// Combinational logic

reg temp;
always @(a or b or c or temp)
begin
    q = temp + c;
    temp = a + b;
end
```



The synthesis standard states that the sensitivity list shall not affect the generation of combinational logic. That means that if the synthesis tool recognizes the block as combinational logic, it will proceed as if you had included all inputs to the logic in the sensitivity list. The synthesis tool may or may not warn you about missing inputs. The generated gates will simulate correctly, but very likely differently than the incorrect RTL simulation.

Modifying RTL Model to Match Synthesis Model

This RTL model embeds testbench constructs to correct the simulation.

```
always @(posedge clk
        or posedge set
        or posedge rst)
    if (set)
        q <= 1;
    else
        if (rst)
            q <= 0;
        else
            q <= d;
```

Best to simply
not do this!



This RTL model stays set if “set” and “rst” are both applied and then only “set” is removed.

```
always @(posedge clk
        or posedge set
        or posedge rst)
    if (set)
        q <= 1;
    else
        if (rst)
            q <= 0;
        else
            q <= d;

`ifndef SYNTHESIS
always @(set or rst)
    if (set)
        assign q = 1;
    else
        if (rst)
            assign q = 0;
        else
            deassign q;
`endif
```

Procedural
Continuous
Assignment

You are unlikely to model both asynchronous set and asynchronous reset, and even less likely to use them in a situation where they are both simultaneously active. If you do, though, the RTL synthesis template will not accurately model the behavior, as it does not respond to the inactive edge of the set or reset inputs.

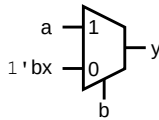
You can work around this with embedded testbench code not meant for synthesis. You can use the procedural continuous assignment to override a normal procedural assignment. You apply a procedural continuous assignment only to a variable, unlike the normal continuous assignment you apply only to a net. While the procedural continuous assignment to a variable is in effect, the simulator ignores any normal procedural assignment to the variable, so be sure to deassign the variable when you want it to again behave normally.

Do Not Confirm Expected Unknown Values

Synthesis of a “x” assignment is a “don’t care” condition that is “optimized away”.

Simulation of an “x” assignment makes the variable value unknown.

```
always @(a or b)
  if (b)
    y = a;
  else
    y = 'bx;
```



```
always @(a or b)
  if (b)
    y = a;
  else
    y = 'bx;
```



Don't confirm x values!

In the real world, the unknown state does not exist. Digital design nodes are almost invariably in a binary state. Synthesis accepts assignment of the x character to indicate a don’t care value.

Assigning the unknown value can help debug your RTL design. You can, for example, initially assign an unknown value to a variable that should always get assigned some other value before it is tested. Just don’t expect the unknown value to show up in simulation of the post-synthesis netlist.

Using casez and Not casex

- Simulation of the **casez** statement treats Z and ? as *don't care* bit positions.
 - In either the case expression or the case item expression.
- Unknown *sel* at start of RTL simulation matches no item.
- Simulation of the **casex** statement treats Z, X and ? as *don't care* bit positions.
 - In either the case expression or the case item expression.
- Unknown *sel* at start of RTL simulation matches first item.

```
always @*
begin
    set = 0;
    rst = 0;
    casex (sel[1:0])
        2'b?1: set = 1;
        2'b1?: rst = 1;
    endcase
end
```

```
always @*
begin
    set = 0;
    rst = 0;
    casez (sel[1:0])
        2'b?1: set = 1;
        2'b1?: rst = 1;
    endcase
end
```



In the real world , the unknown state does not exist and the high-impedance state exists only extremely rarely. Digital design nodes are almost invariably in a binary state.

Synthesis accepts:

- Assignment of the x character to indicate a don't care value;
- Assignment of the z character to infer three-state logic;
- The characters “x”, “z”, and “?” in the casex item expression to designate bit positions to not participate in the match, that is, “don't care” bit positions;
- The characters z, and ? in the casez item expression to designate bit positions to not participate in the match, that is, don't care bit positions;

RTL simulation accepts don't care bit designations also in the case expression. At the start of RTL simulation, much of the design will be in the unknown state and very likely little or none of it in the high-impedance state. The casex statement treats an unknown sel signal as don't care for all bit positions, thus matching the first case item, whatever that might happen to be. The casez statement does a definitive match for the unknown state, which is unlikely to appear in any of the case items, thus leaving the block outputs with their default values.

Avoiding Variable Declaration Assignment

Synthesis ignores variable declaration assignments.

- Hardware generally does not magically power-up into a known state.
- You must provide an explicit initialization path.

```
reg [3:0] count=4'd0;
always @(posedge clk)
  if (count == 9)
    count <= 4'd0;
  else
    count <= count + 4'd1;
```



```
reg [3:0] count;
always @(posedge clk)
  if (rst)
    count <= 4'd0;
  else
    if (count == 9)
      count <= 4'd0;
    else
      count <= count + 4'd1;
```



Synthesis ignores variable declaration assignments. With the exception of some PALs, manufacturers generally do not guarantee that their components power up in any particular state, so variable declaration assignments are rarely truly a hardware construct. As a general statement, you must provide an explicit initialization path for any RTL that you mean to synthesize.

The variable declaration assignment construct is new to the Verilog-2001 update. You may find it useful in your testbench but must not use it in RTL meant for synthesis.

Remembering Delay Controls Are Not Synthesized

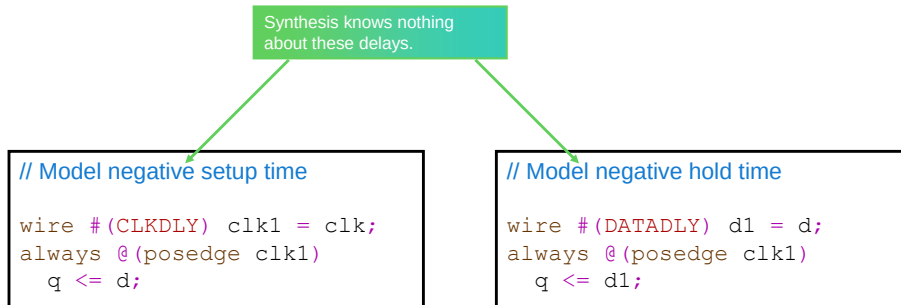
Synthesis *rejects* delay controls before the procedure event control.

Synthesis *ignores* delay controls after the procedure event control.

Synthesis *ignores* net delay and continuous assignment delay.

Time delay is not how you model timing for RTL synthesis.

- You specify arrival and departure times in a separate constraints file.



You can add timing delays to your RTL design to help you visualize the approximate delays of the post-synthesis netlist, but do not for even a moment imagine that this is how you specify the design timing for synthesis. Synthesis totally ignores RTL timing delays that do not violate the synthesis templates. You provide design timing information to synthesis in a separate file as constraints in the vendor's syntax. While simulating the post-synthesis netlist, you may find that some "guesstimates" of the design timing are not as close as you would have liked.

Module Review

1. State and explain the consequence of making an incomplete assignment in a subroutine.
2. State and defend your selection between the **casex** and **casez** statements for designs meant for synthesis.



This page does not contain notes.

Module Review Solutions

1. State and explain the consequence of making an incomplete assignment in a subroutine.
 - Synthesis tools infer a latch if you specify no output value for at least one combination of input values. This latch inference may be inadvertent but it is at least functionally correct. Most synthesis tools do *not* infer such latch if the incomplete assignment is in a subroutine. They instead combinationaly generate an undetermined logic output value for the missing input combination.
2. State and defend your selection between the **casex** and **casez** statements for designs meant for synthesis.
 - Synthesis accepts in a **casex** match expression the "x", "z", and "?" characters, and in a **casez** match expression the "z", and "?" characters to indicate bit positions that do not participate in the match. Simulation accepts these designations also in the **casex** and **casez** expression. At the start of RTL simulation, that expression is likely to be of unknown value, thus matching the *first casex* item and *no casez* item. At the start of post-synthesis simulation, that expression is likely to be of arbitrary value, thus matching *some casex* or **casez** item.



This page does not contain notes.